precisely, let alone to answer adequately. In practice, one is often reduced to trying increasingly larger values of m until one is convinced that satisfactory performance has been obtained for the task at hand. One may improve on trial-and-error schemes with more systematic hyperparameter search methods, such as Bayesian optimization, and in fact this idea has been used to tune the parameters of the deep network used by AlphaZero.

Experience has shown that in many cases the number of required nonlinear units and corresponding dimension of the parameter space can be very large, adding significantly to the difficulty of solving the training problem. This has given rise to many suggestions for modifications of the perceptron structure. An important possibility is to concatenate multiple single layer perceptrons so that the output of the nonlinear layer of one perceptron becomes the input to the linear layer of the next, giving rise to deep neural networks, which we will discuss later.

3.2.1 Training of Neural Networks

Given a set of state-cost training pairs (x^s, β^s), $s = 1, \ldots, q$, the parameters of the neural network A, b, and r are obtained by solving the problem

$$\min_{A,b,r} \sum_{s=1}^{q} \left(\sum_{\ell=1}^{m} r_\ell \sigma \left(\big(Ay(x^s) + b \big)_\ell \right) - \beta^s \right)^2 . \tag{3.12}$$

Note that the cost function of this problem is generally nonconvex, so there may exist multiple local minima.

In practice it is common to augment the cost function of this problem with a *regularization* function, such as a quadratic in the parameters A, b, and r. This is customary in least squares problems in order to make the problem easier to solve algorithmically. However, in the context of neural network training, regularization is primarily important for a different reason: it helps to avoid *overfitting*, which occurs when the number of parameters of the neural network is relatively large (comparable to the size of the training set). In this case a neural network model matches the training data very well but may not do as well on new data. This is a well-known difficulty, which has been researched extensively in machine learning. It is also the subject of much current research, particularly in the context of deep neural networks, as we will discuss later.

An important issue is to select a method to solve the training problem (3.12). While we can use any unconstrained optimization method that is based on gradients, in practice it is important to take into account the cost function structure of problem (3.12). The salient characteristic of this cost function is that it is the sum of a potentially very large number q of component functions. This makes the computation of the cost function value of the training problem and/or its gradient very costly. For this reason

the incremental methods of Section 3.1.3 are universally used for training.† Experience has shown that these methods can be vastly superior to their nonincremental counterparts in the context of neural network training.

The implementation of the training process has benefited from experience that has been accumulated over time, and has provided guidelines for scaling, regularization, initial parameter selection, and other practical issues; we refer to books on neural networks such as Bishop [Bis95], Bishop and Bishop [BiB24], Goodfellow, Bengio, and Courville [GBC16], and Haykin [Hay08], as well as to the overview paper on deep neural network training by Sun [Sun19], and the references quoted therein. Still, incremental methods can be quite slow, and training may be a time-consuming process. Fortunately, the training is ordinarily done off-line, possibly using parallel computation, in which case computation time may not be a serious issue. Moreover, in practice the neural network training problem typically need not be solved with great accuracy. This is also supported by the Newton step view of approximation in value space, which suggests that great accuracy in the off-line training of the terminal cost approximation is not critically important for good performance of the on-line play algorithm.

3.2.2 Multilayer and Deep Neural Networks

An important generalization of the single layer perceptron architecture involves a concatenation of multiple layers of linear and nonlinear functions; see Fig. 3.2.5. In particular the outputs of each nonlinear layer become the inputs of the next linear layer. In some cases it may make sense to add as additional inputs some of the components of the state x or the state encoding $y(x)$.

In the early days of neural networks practitioners tended to use few nonlinear layers (say one to three). For example, Tesauro's backgammon program and its descendants have performed well with one or two nonlinear layers [PaR12]. However, more recently a lot of success in certain problem domains (including image and speech processing, large language models, as well as approximate DP) has been achieved with *deep neural networks*, which involve a considerably larger number of layers.

There are a few questions to consider here. The first has to do with the reason for having multiple nonlinear layers, when a single one is sufficient to guarantee the universal approximation property. Here are some qualitative (and somewhat speculative) explanations:

† The incremental methods are valid for an arbitrary order of component selection within the cycle, but it is common to randomize the order at the beginning of each cycle. Also, in a variation of the basic method, we may operate on a batch of several components at each iteration, called a *minibatch*, rather than a single component. This has an averaging effect, which reduces the tendency of the method to oscillate and allows for the use of a larger stepsize; see the end-of-chapter references.

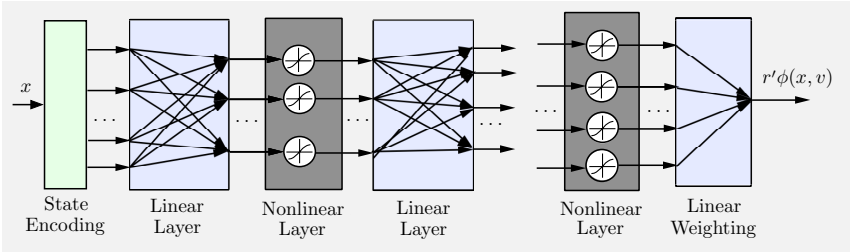

Figure 3.2.5 A deep neural network, with multiple layers. Each nonlinear layer constructs the inputs of the next linear layer.

(a) We may consider the possibility of using linear layers with a particular sparsity pattern, or other structure that embodies special linear operations such as convolution, which may be well-matched to the training problem at hand. Moreover, when such structures are used, the training problem often becomes easier, because the number of parameters in the linear layers is drastically decreased.

(b If we view the outputs of each nonlinear layer as features, we see that the multilayer network produces a hierarchy of features, where each set of features is a function of the preceding set of features [except for the first set of features, which is a function of the encoding $y(x)$ of the state x]. In the context of specific applications, this hierarchical structure can be exploited to specialize the role of some of the layers and to enhance some characteristics of the state.

(c) Overparametrization (more weights than data, as in a deep neural network) helps to mitigate the detrimental effects of overfitting, and the attendant need for regularization. The explanation for this fascinating phenomenon (observed as early as the late 90s) is the subject of much recent research; see [ZBH16], [BMM18], [BRT18], [SJL18], [ADH19], [BLL19], [HMR19], [MVS19], [SuY19], [Sun19], [DMB21], [HaR21], [VLK21], [ZBH21] for representative works.

We finally note that the use of deep neural networks has been an important factor for the success of the AlphaGo and AlphaZero programs, as well as for large language models. New developments in hardware, software, architectural structures, training methodology, and integration with other types of neural networks such as transformers, are likely to improve the already enormous power of deep neural networks, and to allow the use of ever larger datasets, which are becoming increasingly available.

3.3 LEARNING COST FUNCTIONS IN APPROXIMATE DP

In the context of approximate DP/RL, architectures are mainly used to

approximate either cost functions or policies. When a neural network is involved, the terms *value network* and *policy network* are commonly used, respectively.† In this section we will illustrate the use of value networks in finite horizon DP, while in the next section we will discuss the use of policy networks. We will also illustrate in Section 3.3.3 the combined use of policy and value networks within an approximate policy iteration context, whereby the policies and their cost functions are approximated by a policy and a value network, respectively, to generate a sequence of (approximately) improved policies. Finally, in Sections 3.3.6 and 3.3.7, we will describe how approximating Q-factor differences or cost differences (rather than Q-factors or costs) can be beneficial within our context of approximation in value space.

3.3.1 Fitted Value Iteration

Let us describe a popular approach for training an approximation architecture $\tilde{J}_k(x_k, r_k)$ for a finite horizon DP problem. The parameter vectors r_k are determined sequentially, starting from the end of the horizon, and proceeding backwards as in the DP algorithm: first r_{N-1} then r_{N-2}, and so on. The algorithm samples the state space for each stage k, and generates a large number of states x_k^s, $s = 1, \ldots, q$. It then determines sequentially the parameter vectors r_k to obtain a good "least squares fit" to the DP algorithm. The method can also be used in the infinite horizon case, in essentially identical form, and it is commonly called *fitted value iteration*.

In particular, each r_k is determined by generating a large number of sample states and solving a least squares problem that aims to minimize the error in satisfying the DP equation for these states at time k. At the typical stage k, having obtained r_{k+1}, we determine r_k from the least squares problem

$$r_k \in \arg\min_r \sum_{s=1}^{q} \Bigg(\tilde{J}_k(x_k^s, r) - \min_{u \in U_k(x_k^s)} E\Big\{ g_k(x_k^s, u, w_k) + \tilde{J}_{k+1}\big(f_k(x_k^s, u, w_k), r_{k+1}\big) \Big\} \Bigg)^2$$

where x_k^s, $i = 1, \ldots, q$, are the sample states that have been generated for the kth stage. Since r_{k+1} is assumed to be already known, the complicated minimization term in the right side of this equation is the known scalar

$$\beta_k^s = \min_{u \in U_k(x_k^s)} E\Big\{ g_k(x_k^s, u, w_k) + \tilde{J}_{k+1}\big(f_k(x_k^s, u, w_k), r_{k+1}\big) \Big\}, \qquad (3.13)$$

† The alternative terms *critic network* and *actor network* are also used often. In this book, we will use the terms "value network" and "policy network."

so that r_k is obtained as

$$r_k \in \arg\min_r \sum_{s=1}^{q} \big(\tilde{J}_k(x_k^s, r) - \beta_k^s\big)^2. \tag{3.14}$$

The algorithm starts at stage $N-1$ with the minimization

$$r_{N-1} \in \arg\min_r \sum_{s=1}^{q} \Bigg(\tilde{J}_{N-1}(x_{N-1}^s, r) - \min_{u \in U_{N-1}(x_{N-1}^s)} E\Big\{ g_{N-1}(x_{N-1}^s, u, w_{N-1}) + g_N\big(f_{N-1}(x_{N-1}^s, u, w_{N-1})\big)\Big\}\Bigg)^2$$

and ends with the calculation of r_0 at $k = 0$.

In the case of a linear architecture, where the approximate cost-to-go functions are

$$\tilde{J}_k(x_k, r_k) = r_k' \phi_k(x_k), \qquad k = 0, \ldots, N-1,$$

the least squares problem (3.14) greatly simplifies, and admits the closed form solution

$$r_k = \left(\sum_{s=1}^{q} \phi_k(x_k^s) \phi_k(x_k^s)' \right)^{-1} \sum_{s=1}^{q} \beta_k^s \phi_k(x_k^s);$$

cf. Eq. (3.7). For a nonlinear architecture such as a neural network, incremental gradient algorithms may be used.

An important implementation issue is how to select the sample states x_k^s, $s = 1, \ldots, q$, $k = 0, \ldots, N-1$. In practice, they are typically obtained by some form of Monte Carlo simulation, but the distribution by which they are generated is important for the success of the method. In particular, it is important that the sample states are "representative" in the sense that they are visited often under a nearly optimal policy. More precisely, the frequencies with which various states appear in the sample should be roughly proportional to the probabilities of their occurrence under an optimal policy.

Aside from the issue of selection of the sampling distribution that we have just described, a difficulty with fitted value iteration arises when the horizon N is very long, since then the total number of parameters over the N stages may become excessive. In this case, however, the problem is often stationary, in the sense that the system and cost per stage do not change as time progresses. Then it may be possible to treat the problem as one with an infinite horizon and bring to bear additional methods for training approximation architectures; see the relevant discussions in Chapter 5 of the book [Ber19a].

We finally note an important difficulty with the training method of this section: the calculation of each sample β_k^s of Eq. (3.13) requires a minimization of an expected value, which can be very time consuming. In the next section, we describe an alternative type of fitted value iteration, which uses Q-factors, and involves a simpler minimization, whereby the order of the minimization and expectation operations in Eq. (3.13) is reversed.

3.3.2 Q-Factor Parametric Approximation - Model-Free Implementation

We will now consider an alternative form of approximation in value space and fitted value iteration, which involves approximation of the optimal Q-factors of state-control pairs (x_k, u_k) at time k, with no intermediate approximation of cost-to-go functions. An important characteristic of this algorithm is that it *allows for a model-free computation* (i.e., the use of a computer model in place of a mathematical model).

We recall that the optimal Q-factors are defined by

$$Q_k^*(x_k, u_k) = E\Big\{g_k(x_k, u_k, w_k) + J_{k+1}^*\big(f_k(x_k, u_k, w_k)\big)\Big\}, \quad k = 0, \ldots, N-1, \tag{3.15}$$

where J_{k+1}^* is the optimal cost-to-go function for stage $k+1$. Thus $Q_k^*(x_k, u_k)$ is the cost attained by using u_k at state x_k, and subsequently using an optimal policy.

As noted in Section 1.3, the DP algorithm can be written as

$$J_k^*(x_k) = \min_{u \in U_k(x_k)} Q_k^*(x_k, u_k),$$

and by using this equation, we can write Eq. (3.15) in the following equivalent form that relates Q_k^* with Q_{k+1}^*:

$$\begin{aligned} Q_k^*(x_k, u_k) = E\Big\{ & g_k(x_k, u_k, w_k) \\ & + \min_{u \in U_{k+1}(f_k(x_k, u_k, w_k))} Q_{k+1}^*\big(f_k(x_k, u_k, w_k), u\big)\Big\}. \end{aligned} \tag{3.16}$$

This suggests that in place of the Q-factors $Q_k^*(x_k, u_k)$, we may use Q-factor approximations as the basis for suboptimal control.

We can obtain such approximations by using methods that are similar to the ones we have considered so far. Parametric Q-factor approximations $\tilde{Q}_k(x_k, u_k, r_k)$ may involve a neural network, or a feature-based linear architecture. The feature vector may depend on just the state, or on both the state and the control. In the former case, the architecture has the form

$$\tilde{Q}_k(x_k, u_k, r_k) = r_k(u_k)'\phi_k(x_k), \tag{3.17}$$

where $r_k(u_k)$ is a separate weight vector for each control u_k. In the latter case, the architecture has the form

$$\tilde{Q}_k(x_k, u_k, r_k) = r_k' \phi_k(x_k, u_k), \tag{3.18}$$

where r_k is a weight vector that is independent of u_k. The architecture (3.17) is suitable for problems with a relatively small number of control options at each stage. In what follows, we will focus on the architecture (3.18), but the discussion, with few modifications, also applies to the architecture (3.17) and to nonlinear architectures as well.

We may adapt the fitted value iteration approach of the preceding section to compute sequentially the parameter vectors r_k in Q-factor parametric approximations, starting from $k = N - 1$. This algorithm is based on Eq. (3.16), with r_k obtained by solving least squares problems similar to the ones of the cost function approximation case [cf. Eq. (3.14)]. As an example, the parameters r_k of the architecture (3.18) are computed sequentially by collecting sample state-control pairs (x_k^s, u_k^s), $s = 1, \ldots, q$, and solving the linear least squares problems

$$r_k \in \arg\min_r \sum_{s=1}^{q} \big(r' \phi_k(x_k^s, u_k^s) - \beta_k^s\big)^2, \tag{3.19}$$

where

$$\beta_k^s = E\left\{ g_k(x_k^s, u_k^s, w_k) + \min_{u \in U_{k+1}(f_k(x_k^s, u_k^s, w_k))} r_{k+1}' \phi_{k+1}\big(f_k(x_k^s, u_k^s, w_k), u\big) \right\}. \tag{3.20}$$

Thus, having obtained r_{k+1}, we obtain r_k through a least squares fit that aims to minimize the sum of the squared errors in satisfying Eq. (3.16). Note that the solution of the least squares problem (3.19) can be obtained in closed form as

$$r_k = \left(\sum_{s=1}^{q} \phi_k(x_k^s, u_k^s) \phi_k(x_k^s, u_k^s)' \right)^{-1} \sum_{s=1}^{q} \beta_k^s \phi_k(x_k^s, u_k^s);$$

[cf. Eq. (3.7)]. Once r_k has been computed, the one-step lookahead control $\tilde{\mu}_k(x_k)$ is obtained on-line as

$$\tilde{\mu}_k(x_k) \in \arg\min_{u \in U_k(x_k)} \tilde{Q}_k(x_k, u, r_k), \tag{3.21}$$

without the need to calculate any expected value. This latter property is a primary incentive for using Q-factors in approximate DP, particularly when there are tight constraints on the amount of on-line computation that is possible in the given practical setting.

The samples β_k^s of Eq. (3.20) involve the exact computation of an expected value. In an alternative implementation, we may replace β_k^s with an average of just a few samples (even a single sample) of the random variable

$$g_k(x_k^s, u_k^s, w_k) + \min_{u \in U_{k+1}(f_k(x_k^s, u_k^s, w_k))} r'_{k+1}\phi_{k+1}\big(f_k(x_k^s, u_k^s, w_k), u\big), \quad (3.22)$$

collected according to the probability distribution of w_k. This distribution may either be known explicitly, or in a model-free situation, through a computer simulator. In particular, *to implement this scheme, we only need a simulator that for any pair* (x_k, u_k) *generates samples of the next stage cost and state*

$$g_k(x_k, u_k, w_k), \qquad f_k(x_k, u_k, w_k),$$

according to the distribution of w_k.

Note that the samples of the random variable (3.22) do not require the computation of an expected value like the samples (3.13) in the cost approximation method of the preceding chapter. Moreover the samples of (3.22) involve a simpler minimization than the samples (3.13). This is an important advantage of working with Q-factors rather than state costs.

Having obtained the weight vectors $r_0, \ldots, r_{N-1}$, and hence the one-step lookahead policy $\tilde{\pi} = \{\tilde{\mu}_0, \ldots, \tilde{\mu}_{N-1}\}$ through Eq. (3.21), a further possibility is to approximate this policy with a parametric architecture. This is *approximation in policy space built on top of approximation in value space*. The idea here is to simplify even further the on-line computation of the suboptimal controls by avoiding the minimization of Eq. (3.21); see the discussion in Section 1.3.3.

3.3.3 Parametric Approximation in Infinite Horizon Problems - Approximate Policy Iteration

In this section we will briefly discuss parametric approximation methods for infinite horizon problems, based on policy iteration (PI). We will focus on the finite-state version of the α-discounted problem of Section 1.4.1, using notation that is more convenient for such problems. In particular, states and successor states will be denoted by i and j, respectively. Moreover the system equation will be represented by control-dependent transition probabilities $p_{ij}(u)$ (the probability that the system will move to state j, given that it starts at state i and control u is applied). For a state-control pair (i, u), the average cost per stage is denoted by $g(i, u, j)$.

We recall that the PI algorithm in its exact form produces a sequence of stationary policies whose cost functions are progressively improving and converge in a finite number of iterations to the optimal. The convergence proof relies on the fundamental cost improvement property of PI, and depends on the finiteness of the state and control spaces. This proof, together

with other PI-related convergence proofs, can be found in the author's textbooks [Ber17a] or [Ber19a].

Let us state the exact form of the PI algorithm in terms of Q-factors, and in a way that facilitates approximations and simulation-based implementations. Given any policy μ, it generates the next policy $\tilde{\mu}$ with a two-step process as follows (cf. Section 1.4.1):

(a) *Policy evaluation*: We compute the cost function J_μ of μ and its associated Q-factors, which are given by

$$Q_\mu(i,u) = \sum_{j=1}^{n} p_{ij}(u)\big(g(i,u,j) + \alpha J_\mu(j)\big), \qquad i = 1,\ldots,n,\ u \in U(i).$$

Thus $Q_\mu(i,u)$ is the cost of starting at i, using u at the first stage, and then using μ for the remaining stages.

(b) *Policy improvement*: We compute the new policy $\tilde{\mu}$ according to

$$\tilde{\mu}(i) \in \arg\min_{u \in U(i)} Q_\mu(i,u), \qquad i = 1,\ldots,n.$$

Let us now describe one approach to approximating the two steps of the PI process:.

(a) *Approximate policy evaluation*: Here we introduce a parametric architecture $\tilde{Q}_\mu(i,u,r)$ for the Q-factors of μ. We determine the value of the parameter vector r by generating (using a simulator of the system) a large number of training triplets (i^s, u^s, β^s), $s = 1,\ldots,q$, and by using a least squares fit:

$$\overline{r} \in \arg\min_r \sum_{s=1}^{q} \big(\tilde{Q}_\mu(i^s,u^s,r) - \beta^s\big)^2. \tag{3.23}$$

In particular, for a given pair (i^s, u^s), the scalar β^s is generated by starting at i^s, using u^s at the first stage, and simulating a trajectory of states and controls using μ for some number k of subsequent stages. Thus, β^s is a sample of $Q_\mu^k(i^s,u^s)$, the k-stage Q-factor of μ, which in the limit as $k \to \infty$ yields the infinite horizon Q-factor of μ. The number of stages k may be either large, or fairly small. However, in the latter case some terminal cost function approximation should be added at the end of the k-stage trajectory, to compensate for the difference $\big|Q_\mu(i,u) - Q_\mu^k(i,u)\big|$, which decreases in proportion to α^k, and may be large when k is small. Such a function may be obtained with additional training or from a previous iteration.

(b) *Approximate policy improvement*: Here we compute the new policy $\tilde{\mu}$ according to

$$\tilde{\mu}(i) \in \arg\min_{u \in U(i)} \tilde{Q}_\mu(i,u,\overline{r}), \qquad i = 1,\ldots,n, \tag{3.24}$$